Step 2 — Saved State and a Human Pause Before Anything Goes Out — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: a LangGraph graph can do something a hand-written loop can't: **stop in the middle, save everything to disk, and continue later** — even after the program was closed and restarted. The mechanism is `interrupt()` inside a node plus a checkpointer that persists state outside the Python process. That's what lets a real person approve an answer before it goes out. This step adds the Approval agent's file and node — `approval_node`, which for now asks a person about **every** answer — plus a checkpointer and a thread id per run. **If not:** Step 5 would have to build the pause, the saved state and the node all at the same time as its real approval checks.

After this step every answer waits for your yes/no, and a paused run survives a restart. `answer_node` and `clean_task` don't change. Files not shown here don't change.

Solution — the Approval agent's node, a SQLite checkpointer, and a real restart test

Changed from Step 1: `state.py` adds `approved`.

```
# src/state.py
# DocState: everything any node can read or write. `path` adds each node's
# entries to the list (operator.add), so it logs the nodes that ran.
# start_state(task) gives every field a starting value.
import operator
from typing import Annotated, TypedDict

class DocState(TypedDict):
    task: str
    path: Annotated[list[str], operator.add]
    # new in Step 2: whether a person (or later, the checks) approved it
    approved: bool
    answer: str

def start_state(task):
    return {"task": task, "path": [], "approved": False, "answer": ""}
```

Story — `agents/approval_agent.py` : this is the Approval agent's file, with its node present from day one. `approval_node` calls `interrupt()`, which genuinely pauses graph execution right there and checkpoints the state; whatever value you later pass to `Command(resume=...)` comes back as that call's return value — here captured as `decision` (Doc09's pattern). For now its list of `reasons` is one fixed line, so **every** answer goes to a person; Step 5 replaces that one line with real checks that decide when review is actually needed. (Also create an empty `agents/__init__.py`.) **If not:** answers would go out with no human check at all.

```
# src/agents/approval_agent.py
# The Approval agent: approval_node() pauses with interrupt() and waits for
# a person's yes or no. On resume, the node runs again from its first line,
# so nothing above interrupt() may do a real action.
from langgraph.types import interrupt

from src.utils.logger import get_logger

def approval_node(state):
    logger = get_logger(__name__)
    reasons = ["every answer is checked by a person for now"]

    logger.info(f"asking a human, failed checks: {reasons}")
    decision = interrupt({"answer": state["answer"], "failed_checks": reasons})
    if decision == "yes":
        return {"approved": True, "path": ["human_approved"]}
    return {"approved": False,
            "answer": "Not approved — nothing was sent.",
            "path": ["human_rejected"]}
```

Changed from Step 1: a third node, and `build_graph()` now takes a checkpointer.

Story — `graph.py` : the checkpointer is **passed in** as a parameter to `build_graph()` (Doc09's pattern), not hard-coded inside it: `main.py` passes a real `SqliteSaver` that writes to disk, while tests pass a `MemorySaver` that lives only in RAM and leaves no files behind. **If not:** the checkpointer would be hard-coded, and a test and the real app couldn't each use the kind that suits them.

```
# src/graph.py
# build_graph(): adds the nodes and edges and compiles the graph. The
# checkpointer is passed in, so the app and the tests can each choose one.
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

from src.agents.approval_agent import approval_node
from src.nodes import answer_node, clean_task
from src.state import DocState
```

```
def build_graph(checkpointer=None):
    if checkpointer is None:
        checkpointer = MemorySaver()

    builder = StateGraph(DocState)
    builder.add_node("clean_task", clean_task)
    builder.add_node("answer_node", answer_node)
    builder.add_node("approval_node", approval_node)

    builder.add_edge(START, "clean_task")
    builder.add_edge("clean_task", "answer_node")
    builder.add_edge("answer_node", "approval_node")
    builder.add_edge("approval_node", END)
    return builder.compile(checkpointer=checkpointer)
```

Changed from Step 1: `main.py` opens a `SqliteSaver`, gives each question its own thread id, and handles the pause with `ask_human()`.

Story — `main.py`: `SqliteSaver.from_conn_string(DB_PATH)` stores every checkpoint in a real `checkpoints.sqlite` file (add it to `.gitignore`), not in memory. Each question gets its own `thread_id`, which is what keeps separate runs' saved states from colliding. When the graph's result contains `"__interrupt__"`, `ask_human()` shows the answer and the reason, asks yes/no, and the app resumes the exact paused run with `Command(resume=decision)`. **If not:** a paused run would be lost on restart, and two questions could share one saved state.

```
# main.py
# The file you run: python main.py
# Loads the settings, builds the graph, then answers questions in a loop,
# printing the path each run took.
import sys
import time

import openai
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.state import start_state
from src.utils.logger import get_logger

DB_PATH = "checkpoints.sqlite"

def ask_human(request):
```

```
    print("  -- needs your approval --")
    print(f"  answer: {request['answer']}")
    print(f"  why:   {', '.join(request['failed_checks'])}")
    return input("  Approve? (yes/no): ").strip().lower()

def main():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    print("DocuMind — ask a question, or /quit to exit.")

    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        graph = build_graph(checkpointer)
        run_number = 0
        while True:
            task = input("You: ").strip()
            if task == "/quit":
                break
            if task == "":
                continue

            run_number = run_number + 1
            thread_id = f"run-{int(time.time())}-{run_number}"
            config = {"configurable": {"thread_id": thread_id}}
            try:
                result = graph.invoke(start_state(task), config)
                if "__interrupt__" in result:
                    request = result["__interrupt__"][0].value
                    decision = ask_human(request)
                    result = graph.invoke(Command(resume=decision), config)
            except openai.AuthenticationError:
                logger.error("Authentication failed — check OPENAI_API_KEY.")
                print("Login failed — check OPENAI_API_KEY in your .env file.")
                sys.exit(1)
            except openai.RateLimitError:
                print("We're busy right now — wait a moment and try again.")
                continue

            print(f"  path: {' -> '.join(result['path'])}")
            print(f"Answer: {result['answer']}")

if __name__ == "__main__":
    main()
```

Story — `scripts/pause_demo.py` + `scripts/resume_demo.py`: this is the real proof the pause survives a restart, not just a claim. Part 1 starts a run, stops at the `interrupt()`, and then the process **exits entirely** — no Python objects survive. Part 2 is run later, as a genuinely **separate process**: it finds the paused run sitting in the SQLite file using `graph.get_state(config)` and finishes it with `Command(resume=...)`. (Doc09 showed `MemorySaver` fails exactly this test, because its state lives only in RAM.) **If not:** "it resumes" might only mean "the objects were still in memory," which isn't what a real pause-for-approval feature needs.

```
# scripts/pause_demo.py
# Part 1 of the restart proof: starts a run that needs approval, saves the
# pause to checkpoints.sqlite, and exits.
# Run from the repo root: python -m scripts.pause_demo
from langgraph.checkpoint.sqlite import SqliteSaver

from src.config import load_config
from src.graph import build_graph
from src.state import start_state

DB_PATH = "checkpoints.sqlite"
THREAD_ID = "demo-1"
TASK = "What is the refund policy? Send it to the customer."

def main():
    load_config()
    config = {"configurable": {"thread_id": THREAD_ID}}
    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        graph = build_graph(checkpointer)
        result = graph.invoke(start_state(TASK), config)
    if "__interrupt__" in result:
        request = result["__interrupt__"][0].value
        print(f"Paused on thread {THREAD_ID}, waiting for approval:")
        print(f"  answer: {request['answer']}")
        print("Now close this and run: python -m scripts.resume_demo")
    else:
        print("The run finished without pausing — nothing to resume.")

if __name__ == "__main__":
    main()
```

```
# scripts/resume_demo.py
# Part 2: a brand-new process finds the paused run in checkpoints.sqlite
# with get_state() and finishes it.
# Run from the repo root: python -m scripts.resume_demo
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command
```

```
from src.config import load_config
from src.graph import build_graph

DB_PATH = "checkpoints.sqlite"
THREAD_ID = "demo-1"

def main():
    load_config()
    config = {"configurable": {"thread_id": THREAD_ID}}
    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        graph = build_graph(checkpointer)
        saved = graph.get_state(config)
        if not saved.next:
            print(f"Nothing is paused on thread {THREAD_ID}.")
            return
        print(f"Found a paused run: {saved.values['task']}")
        decision = input("Approve? (yes/no): ").strip().lower()
        result = graph.invoke(Command(resume=decision), config)
    print(f"path: {' -> '.join(result['path'])}")
    print(f"Answer: {result['answer']}")

if __name__ == "__main__":
    main()
```

Changed from Step 1: `tests/integration/test_live_graph.py` gets cases 2-4. `run()` gains one optional parameter, `decision`: if the run paused, it resumes with that answer (`None` means "leave it paused"). The test builds the graph with a fresh `MemorySaver`, so nothing is written to disk.

Story — `tests/integration/test_live_graph.py`: the pause is this step's headline feature, so it gets three checks against the real model and a real `MemorySaver`: a task pauses at the interrupt, resuming with "yes" approves and returns the answer, and resuming with "no" stops it from going out. **If not:** a pause that didn't actually stop anything — or a "no" that still silently sent the answer — would go unnoticed.

```
# tests/integration/test_live_graph.py
# Runs the whole graph against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_graph
# new in Step 2
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.state import start_state
```

```
# new in Step 2
ACTION_TASK = "What is the refund policy? Send it to the customer."


# changed in Step 2: resumes a paused run with `decision` (None = don't)
def run(graph, thread_id, task, decision="yes"):
    config = {"configurable": {"thread_id": thread_id}}
    outcome = graph.invoke(start_state(task), config)
    if decision is not None and "__interrupt__" in outcome:
        outcome = graph.invoke(Command(resume=decision), config)
    return outcome


def show(case, passed, outcome):
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}: {outcome.get('path')}")


def test_general_question(graph):
    outcome = run(graph, "t1", "What is the capital of France?")
    passed = "Paris" in outcome["answer"] and "answer_node" in outcome["path"]
    show("case 1 (general question)", passed, outcome)


# new in Step 2
def test_action_pauses(graph):
    outcome = run(graph, "t2", ACTION_TASK, decision=None)
    show("case 2 (pauses)          ", "__interrupt__" in outcome, outcome)


# new in Step 2
def test_human_says_yes(graph):
    outcome = run(graph, "t3", ACTION_TASK, "yes")
    passed = outcome["approved"] and "human_approved" in outcome["path"]
    show("case 3 (human: yes)      ", passed, outcome)


# new in Step 2
def test_human_says_no(graph):
    outcome = run(graph, "t4", ACTION_TASK, "no")
    passed = outcome["answer"].startswith("Not approved")
    show("case 4 (human: no)       ", passed, outcome)
```

```
if __name__ == "__main__":
    load_config()
    # changed in Step 2: a fresh MemorySaver, nothing on disk
    graph = build_graph(MemorySaver())
    test_general_question(graph)
    # new in Step 2
    test_action_pauses(graph)
    test_human_says_yes(graph)
    test_human_says_no(graph)
```

Expected output of `python -m tests.integration.test_live_graph` (every answer pauses in this step, and case 1 is resumed with "yes"):

```
case 1 (general question) -> OK: ['clean_task', 'answer_node',
'human_approved']
case 2 (pauses)           -> OK: ['clean_task', 'answer_node']
case 3 (human: yes)       -> OK: ['clean_task', 'answer_node',
'human_approved']
case 4 (human: no)        -> OK: ['clean_task', 'answer_node',
'human_rejected']
```

(Wrapped to fit the page.)

Run it:

```
python -m scripts.pause_demo      # pauses, then exits
python -m scripts.resume_demo     # a new process finishes the run
python main.py
```

Expected output (the answer's wording varies):

```
$ python -m scripts.pause_demo
Paused on thread demo-1, waiting for approval:
  answer: Refund policies usually allow returns within 14-30 days ...
Now close this and run: python -m scripts.resume_demo

$ python -m scripts.resume_demo
Found a paused run: What is the refund policy? Send it to the customer.
Approve? (yes/no): yes
path: clean_task -> answer_node -> human_approved
Answer: Refund policies usually allow returns within 14-30 days ...
```

Notice the answer is a *general* guess — the graph can't read your documents yet. That's Step 3.